

TECHNICAL DOCUMENT

ECOM-12

E-Commerce Orchestration System

Technical Operations Manual

Architectural and Operational Specification

Document Code: DOC-ECOM12-001

Version: 1.0

Date: February 16, 2026

Classification: INTERNAL - COMPANY USE ONLY

Status: APPROVED

KNOWLEDGE TRANSFER DOCUMENT

Revision History

Version	Date	Author	Change Description
1.0	02/16/2026	Engineering Team	Initial document release

Approval Register

Role	Name	Date	Signature
Prepared by			
Reviewed by			
Approved by			

Table of Contents

1. System Introduction

1.1 Document Purpose

This document constitutes the complete technical operational specification for the ECOM-12 (E-Commerce Orchestration System). Its primary purpose is to provide a detailed description of the software architecture, configuration parameters, temporal dependencies, and operational dynamics of the system, in order to support development, maintenance, testing, and operational management activities.

This document has been prepared with particular attention to knowledge transfer, thus representing a fundamental resource for onboarding new technical personnel and ensuring operational continuity of the system.

1.2 Scope of Application

The ECOM-12 system represents a complex software architecture specifically designed for the orchestration and management of a high-throughput e-commerce processing pipeline. The system was conceived to operate in a business-critical environment where reliability, scheduling precision, and efficient use of computational resources are fundamental requirements for operational success.

The system architecture is based on a distributed execution model that coordinates multiple specialized services, each responsible for specific business functions such as payment processing, inventory management, and order fulfillment. These services orchestrate the execution of interdependent tasks that must comply with strict timing constraints and well-defined precedence relationships.

1.3 Intended Audience

This document is intended for: software engineers responsible for system development and maintenance, operations teams responsible for monitoring and control, quality assurance teams for verification and validation activities, training personnel for knowledge transfer programs, and technical management for project activity supervision.

2. Global Configuration and Operational Parameters

2.1 Operational Time Window

The ECOM-12 system operates within a well-defined time window extending from the initial instant, identified as time zero, up to time unit two hundred and ten. This operational window of two hundred and ten time units represents the complete processing cycle during which all scheduled operations must be started, executed, and completed.

The choice of this specific duration is closely linked to the service level agreements (SLAs) and the expected transaction volume, ensuring that all critical order processing stages can be completed within the defined performance constraints.

2.2 System Parameters

Parameter	Value	Description
System Name	12tasks	Unique instance identifier
Operational Window (t_max)	210 t.u.	Maximum operational cycle duration
Max Resources (r_max)	130	Simultaneous processing units
Active Services	6	Number of service modules
Total Tasks	12	Number of operational tasks

The availability of one hundred and thirty resources allows for a massive degree of parallelism in task execution, enabling the system to effectively exploit the processing capabilities of the underlying infrastructure. This high resource count is essential for handling peak loads and ensuring the timely completion of numerous concurrent operations within the defined processing window.

3. Modular Service Architecture

3.1 Architecture Overview

The ECOM-12 system architecture is structured into six main services, each designed to manage a specific functional domain of the e-commerce pipeline. This modular, service-oriented organization enables a clear separation of responsibilities and facilitates both maintenance and system evolution over time.

ID	Service	Primary Responsibility
1	Payment_Service	Manages all financial transaction aspects, including validation and tax calculation
2	Inventory_Service	Handles stock level verification, warehouse allocation, and quality control
3	Order_Service	Orchestrates the core order lifecycle from confirmation to archival
4	Shipping_Service	Manages logistics calculations for order fulfillment
5	Notification_Service	Responsible for all outbound customer communications
6	Security_Service	Ensures regulatory and security compliance checks are performed

3.2 Detailed Service Descriptions

3.2.1 Payment_Service (ID: 1)

The Payment_Service is responsible for the financial integrity of transactions. It manages two critical tasks: `Payment_Validation`, which authenticates and secures payment details, and `Tax_Calculation`, which applies the correct tax rules. `Payment_Validation` runs for 5 time units, requiring 4 resources, and can execute up to 4 instances concurrently. `Tax_Calculation` is a repetitive task, executing 4 times with an 8 time unit delay between starts, requiring 2 resources per instance.

3.2.2 Inventory_Service (ID: 2)

The Inventory_Service manages the physical stock. It orchestrates three tasks: `Inventory_Check`, a repetitive task that verifies stock availability 4 times, requiring 5 resources for its 4 time unit duration; `Warehouse_Allocation`, which assigns stock from a specific location, running for 3 time units with 2 resources; and `Quality_Check`, a 7 time unit task that ensures product standards are met, requiring a single resource.

3.2.3 Order_Service (ID: 3)

The Order_Service is the core of the processing pipeline, managing the order from confirmation to long-term storage. It includes four distinct tasks: Order_Confirmation (duration 8, resources 2), Final_Processing (duration 10, resources 4, repeats 3 times), Document_Generation (duration 9, resources 5, repeats 3 times), and Archive_Processing (duration 4, resources 2). This service demonstrates the full lifecycle management within the system.

3.2.4 Shipping_Service (ID: 4)

The Shipping_Service is a specialized module focused exclusively on logistics. Its sole task, Shipping_Calculation, is a resource-intensive, repetitive operation that runs 4 times. Each execution lasts 10 time units and consumes 3 resources, reflecting the complexity of calculating shipping costs and routes for potentially numerous concurrent orders.

3.2.5 Notification_Service (ID: 5)

The Notification_Service is responsible for customer communication. It contains a single repetitive task, Customer_Notification. This task executes 4 times to ensure timely updates to the customer, with each execution lasting 9 time units and requiring 3 computational resources. This service is critical for customer satisfaction and transparency.

3.2.6 Security_Service (ID: 6)

The Security_Service ensures all operations adhere to regulatory and security standards. It executes the Compliance_Check task, a repetitive process that runs 3 times. Each check lasts 5 time units and requires 5 resources. A critical feature of this task is its max_c of 1, ensuring that compliance checks are performed serially, preventing any potential race conditions in security validation.

4. Detailed Description of Operational Tasks

4.1 Complete Task Registry

ID	Task Code	Service	Duration	Resources	Max Concur.	Rep (rc/rd)
1	PAY_VAL	Payment_Service	5	4	4	0/0

ID	Task Code	Service	Duration	Resources	Max Concur.	Rep (rc/rd)
2	INV_CHK	Inventory_Service	4	5	5	3/0
3	SHIP_CALC	Shipping_Service	10	3	3	3/0
4	ORD_CONF	Order_Service	8	2	2	0/0
5	CUST_NOTIF	Notification_Service	9	3	3	3/0
6	WH_ALLOC	Inventory_Service	3	2	4	0/0
7	QC_CHK	Inventory_Service	7	1	2	0/10
8	FINAL_PROC	Order_Service	10	4	3	2/0
9	TAX_CALC	Payment_Service	5	2	5	3/8
10	DOC_GEN	Order_Service	9	5	2	2/0
11	COMP_CHK	Security_Service	5	5	1	2/0
12	ARCH_PROC	Order_Service	4	2	5	0/13

Legend: Duration = time units; Resources = required computational units; Max Concur. = maximum concurrent executions; Rep = repetitions (rc/rd format where rc = repeat count, rd = repeat delay). Note: `rc=0` indicates a single execution.

The system exhibits a highly heterogeneous task profile, with resource requirements (`res_q`) ranging from 1 to 5 units and durations from 3 to 10 time units. Numerous tasks are configured for repetitive execution, some with specified delays (`rd`) and others without, indicating immediate re-execution. The total available resources of 130 units allow for significant concurrent execution, bounded by the `max_c` limit of each individual task.

5. Temporal Constraints and Dependency System

5.1 Start-to-Start Precedence Model

The ECOM-12 system implements a sophisticated temporal constraint system based on start-to-start (SS) precedence. This constraint type dictates that a destination task can only begin its execution after a specified delay has elapsed since the start of a source task.

This system employs a critical flag, `wait_all`, which modifies the behavior of constraints involving repetitive source tasks:

* `wait_all: false`: The destination task is eligible to start after the delay has passed from the beginning of the *first* execution of the source task. This allows for immediate pipelining and parallel execution flows.

* `wait_all: true`: The destination task must wait until *all* repetitions of the source task are complete. The start time eligibility is still calculated from the start of the *first* execution of the source task, but is gated by the completion of the final repetition. This creates a synchronization barrier, ensuring a complete process cycle finishes before its dependent tasks begin.

5.2 Dependency Matrix

Source Task	Destination Task	Delay (t.u.)	Type	Wait All
Payment_Validation (1)	Inventory_Check (2)	5	SS	false
Inventory_Check (2)	Shipping_Calculation (3)	9	SS	true
Payment_Validation (1)	Order_Confirmation (4)	21	SS	false
Inventory_Check (2)	Order_Confirmation (4)	16	SS	true
Shipping_Calculation (3)	Order_Confirmation (4)	10	SS	true
Order_Confirmation (4)	Customer_Notification (5)	11	SS	false
Shipping_Calculation (3)	Warehouse_Allocation (6)	16	SS	true
Payment_Validation (1)	Warehouse_Allocation (6)	15	SS	false
Customer_Notification (5)	Warehouse_Allocation (6)	15	SS	false
Shipping_Calculation (3)	Quality_Check (7)	11	SS	true
Payment_Validation (1)	Final_Processing (8)	16	SS	false
Warehouse_Allocation (6)	Final_Processing (8)	13	SS	true
Inventory_Check (2)	Tax_Calculation (9)	9	SS	true
Quality_Check (7)	Tax_Calculation (9)	7	SS	false
Warehouse_Allocation (6)	Tax_Calculation (9)	7	SS	true
Inventory_Check (2)	Document_Generation (10)	9	SS	true
Tax_Calculation (9)	Document_Generation (10)	5	SS	true
Order_Confirmation (4)	Document_Generation (10)	14	SS	true
Inventory_Check (2)	Compliance_Check (11)	13	SS	true

Source Task	Destination Task	Delay (t.u.)	Type	Wait All
Warehouse_Allocation (6)	Archive_Processing (12)	9	SS	false

5.3 Critical Dependency Analysis

The dependency network is complex and reveals several key architectural patterns. `Payment_Validation` (1) and `Inventory_Check` (2) act as primary entry points, initiating multiple parallel processing branches.

A significant convergence point is `Order_Confirmation` (4), which depends on the completion of `Payment_Validation` (1), `Inventory_Check` (2), and `Shipping_Calculation` (3). The use of `wait_all: true` for the dependencies from the repetitive tasks (2 and 3) ensures that a full inventory check and shipping calculation cycle is completed before an order can be confirmed, representing a critical synchronization point in the workflow.

Similarly, tasks like `Tax_Calculation` (9) and `Document_Generation` (10) act as integration points, depending on multiple upstream processes. The mixed use of `wait_all` flags creates a sophisticated flow, where some paths are tightly synchronized (`wait_all: true`), while others are loosely coupled and designed for high throughput (`wait_all: false`). For example, `Warehouse_Allocation` (6) can be triggered early by `Payment_Validation` or `Customer_Notification` but must wait for the full `Shipping_Calculation` cycle if following that path.

6. Final Considerations and Recommendations

6.1 Architecture Summary

The ECOM-12 system is a powerful and flexible service-oriented architecture designed for complex, high-volume transaction processing. Its design effectively balances the need for parallel execution to maximize throughput with the requirement for strict synchronization points to ensure data integrity and logical consistency.

The generous allocation of computational resources (130 units), combined with a heterogeneous mix of single-run and repetitive tasks governed by a nuanced

dependency model, demonstrates a mature approach to designing resilient and performant business-critical systems.

6.2 Modular Architecture Benefits

The modularity of the architecture, with its clear separation into specialized services, is a key strength. This design facilitates system comprehension, simplifies maintenance, and enables agile evolution. New functionalities or alternative service implementations (e.g., a new shipping provider) can be integrated with minimal impact on the overall system, providing significant long-term business advantages.

The sophisticated use of the `wait_all` flag in temporal constraints provides fine-grained control over the operational flow, allowing system architects to precisely define the desired balance between process speed and synchronization.

6.3 Knowledge Transfer Recommendations

For effective knowledge transfer, it is recommended to follow a structured training path that includes: understanding global parameters and the operational processing window; studying the service architecture and the responsibilities of each of the six services; detailed analysis of the 12 operational tasks, noting their diverse resource, concurrency, and repetition characteristics; mapping the complete temporal dependency network, with a strong focus on the differing semantics of `wait_all: true` and `wait_all: false` constraints; and simulating operational flows to understand the critical synchronization points (e.g., at `Order_Confirmation`) and the parallel execution pathways.

Appendix A: Glossary

Term	Definition
SOA	Service-Oriented Architecture - A software design paradigm based on distinct functional units (services)
t.u.	Time Unit - Base unit for time measurement in the system
r_max	Maximum number of computational resources allocable simultaneously
t_max	Maximum duration of operational window in time units
Start-to-Start (SS)	Type of precedence constraint based on task start rather than completion

Term	Definition
wait_all	Constraint flag that modifies dependency logic for repetitive tasks, forcing a wait for all repetitions to complete if true
Periodic Task	Task configured to execute multiple repetitions at regular intervals
rc	Repeat Count - Number of additional repetitions for a task (rc=0 means one execution)
rd	Repeat Delay - Time delay between the starts of consecutive repetitions
max_c	Maximum Concurrency - Maximum number of concurrent executions allowed for a task
Pipeline	A set of data processing elements connected in series, where the output of one element is the input of the next

Appendix B: Reference Documents

Code	Document Title	Version
REF-001	ECOM-12 System Requirements Specification	[To be filled]
REF-002	Interface Control Document - Payment Gateway	[To be filled]
REF-003	Verification and Validation Plan	[To be filled]
REF-004	System Operations Manual	[To be filled]

END OF DOCUMENT